MÓDULO 6 · APRENDIZADO NÃO SUPERVISIONADO

Aula 6.10

K-Means do zero

Distâncias, médias, centroides e inércia: como o algoritmo encontra grupos em dados sem rótulos, implementado com NumPy.

120 min Python · NumPy · Matplotlib Exercício: segmentação de clientes

BLOCO 1 · OBJETIVOS

Ao final da aula, você conseguirá

Conceitos

Diferenciar supervisionado e não supervisionado

Explicar clusterização e o K-Means

Entender o significado de K

Identificar um centroide

Diferenciar K-Means de KNN

Matemática à mão

Calcular centroides e distâncias

Atribuir amostras aos clusters

Atualizar centroides com médias

Reconhecer convergência

Calcular a inércia e ler o cotovelo

Implementação

Implementar K-Means com NumPy

Avaliar semente e escalonamento

Visualizar os clusters encontrados

Interpretar os grupos

Comparar com o Scikit-learn

ROTEIRO · 120 MINUTOS

Do papel ao NumPy

00–10	Objetivos e recapitulação	10 min
10–30	Aprendizado não supervisionado e K-Means	20 min
30–55	Centroides, distâncias e cálculo manual	25 min
55–70	Iterações, convergência e inércia	15 min
70–100	Implementação guiada, do notebook vazio	30 min
100–115	Visualização e interpretação	15 min
115–120	Quiz e desafio final	5 min

BLOCO 2 · RECAPITULAÇÃO

Até aqui, todos os modelos aprenderam com rótulos

Regressão Logística

Ajusta pesos para estimar a probabilidade da classe

KNN

Consulta os K vizinhos rotulados mais próximos

Naive Bayes

Estima probabilidades condicionais por classe

Árvore de Decisão

Escolhe divisões que separam as classes

`X`

Características

colunas que descrevem cada amostra

`y`

Rótulos

a resposta correta de cada amostra

`fit(X, y)`

Treinamento

ajusta o modelo comparando com y

`predict(X)`

Predição

devolve uma classe para amostras novas

BLOCO 2 · A PERGUNTA

E se tivéssemos os dados, mas não soubéssemos quais são as classes?

cliente	compras/mês	gasto/compra	rótulo
#001	1,7	R$ 38,89	?
#052	9,1	R$ 68,32	?
#113	5,9	R$ 229,48	?
…	…	…	…

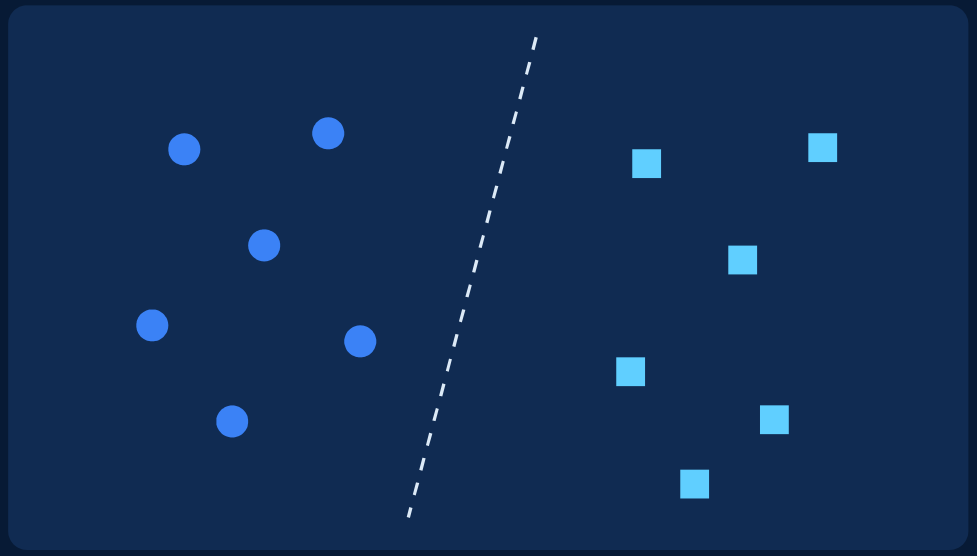
Uma loja registra a **frequência de compras** e o **gasto médio por compra** de cada cliente.

Não existe a coluna «cliente frequente» ou «cliente ocasional».

Podemos procurar grupos de clientes semelhantes?

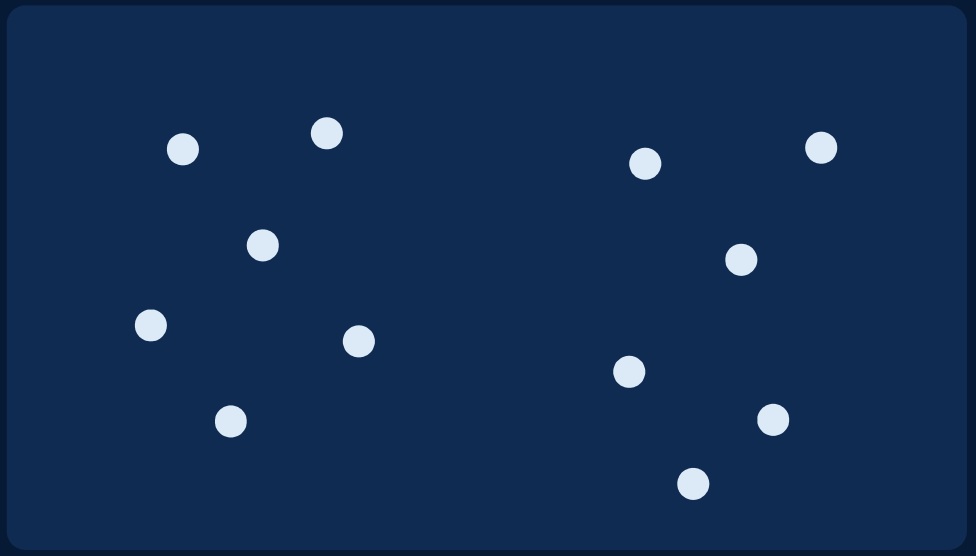
BLOCO 2 · APRENDIZADO NÃO SUPERVISIONADO

Classificação × clusterização



Supervisionado

Usa características **e rótulos conhecidos**. Aprende a prever a classe.

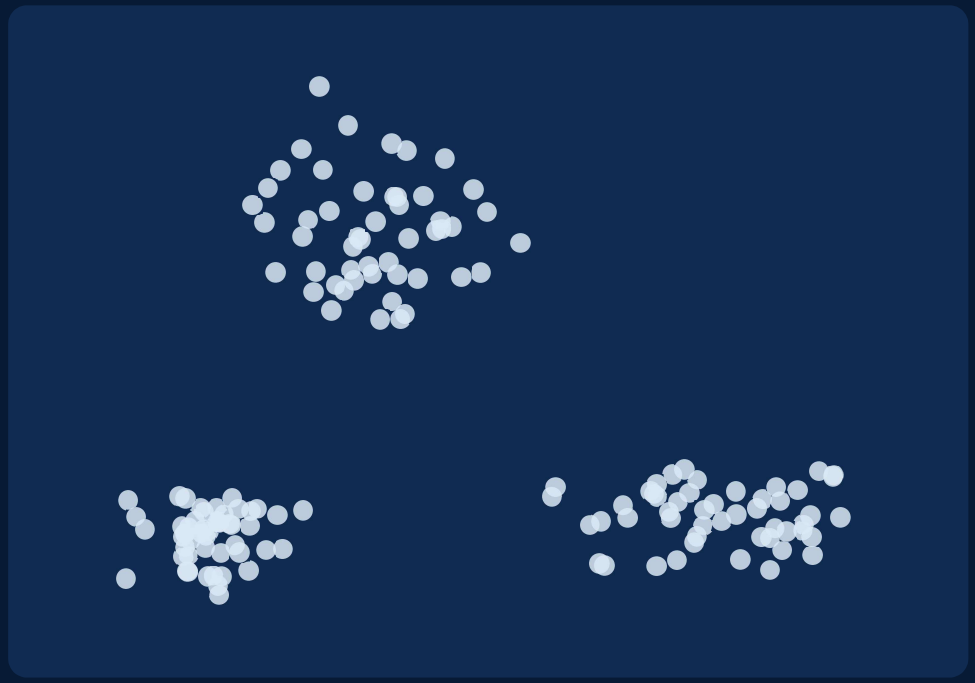


Não supervisionado

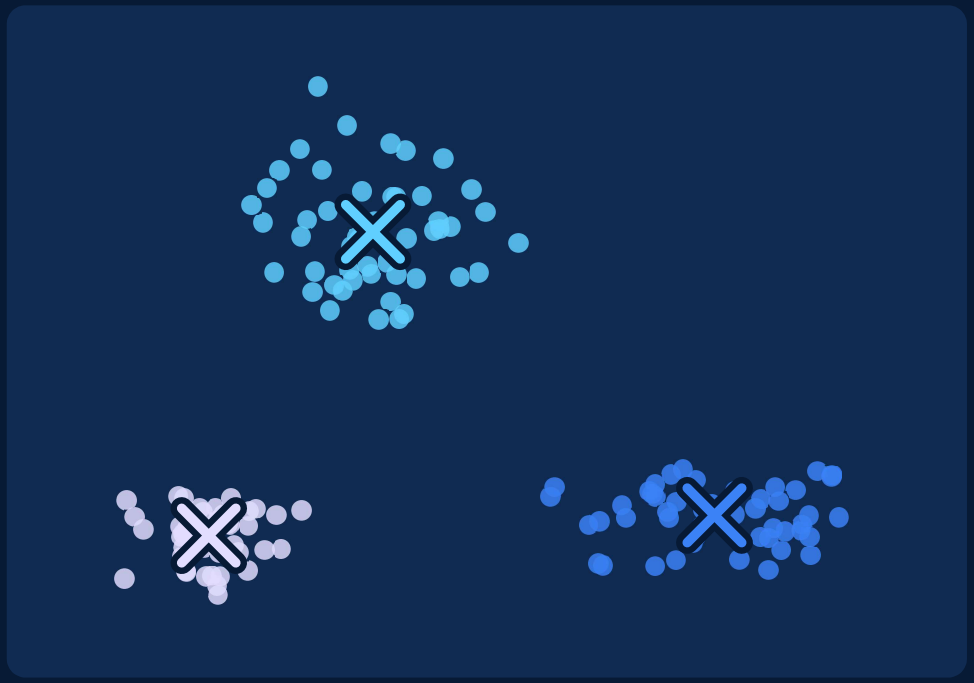
Procura padrões **sem rótulos de resposta** durante o ajuste. O K-Means agrupa por proximidade.

BLOCO 2 · DADOS SEM RÓTULOS

Os mesmos pontos, antes e depois do agrupamento



Entrada: 150 clientes, nenhum rótulo · eixos padronizados



Saída do K-Means (K = 3): cores = clusters, X = centroides

BLOCO 3 · O QUE É K-MEANS?

K grupos, cada um representado por uma média

K

Quantidade de clusters

Número inteiro escolhido antes do treino. O algoritmo não o descobre sozinho.

Means

Médias

Cada grupo é resumido pela média das coordenadas dos seus pontos.

Cluster

Grupo de amostras

Amostras atribuídas ao mesmo centroide.

Centroide

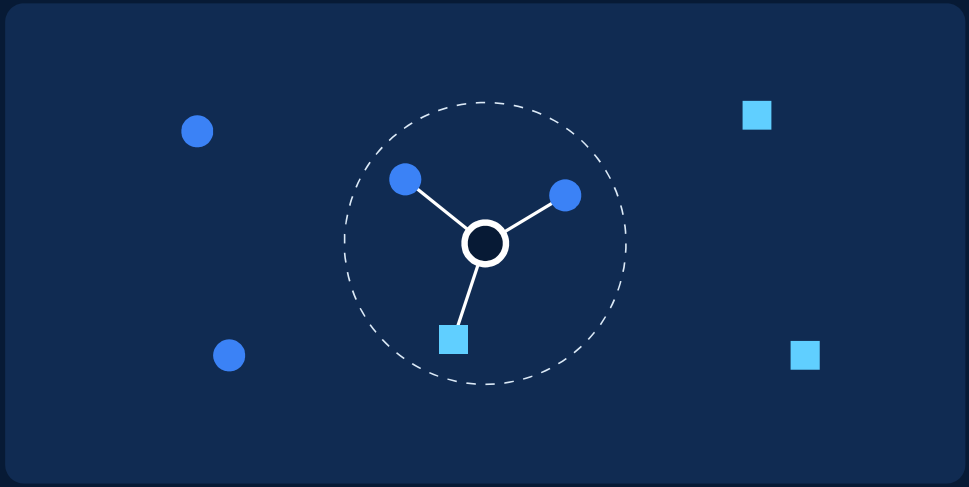
Centro do grupo

Ponto cujas coordenadas são as médias do cluster.

Objetivo: posicionar K centroides de modo que cada amostra fique o mais perto possível do centroide do seu grupo.

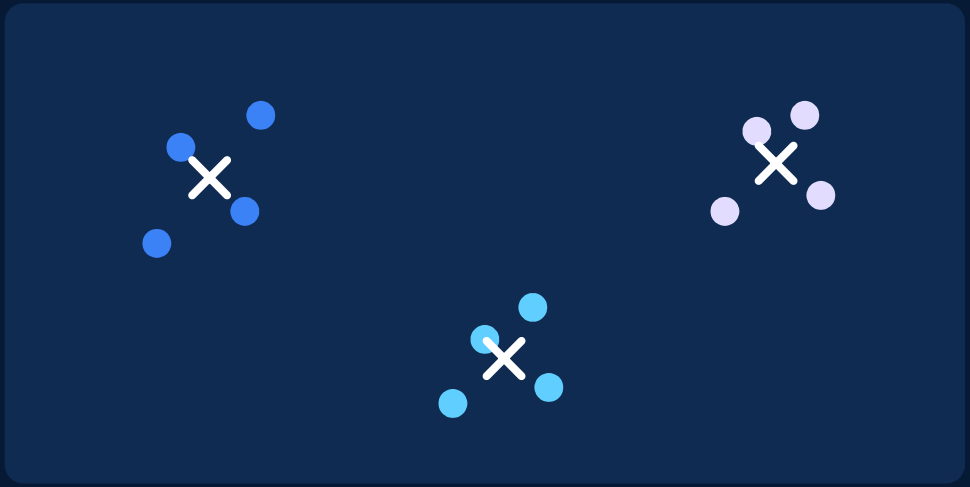
BLOCO 3 · NÃO CONFUNDA

KNN e K-Means: mesmo K, significados diferentes



KNN · K = vizinhos consultados

Supervisionado · precisa de rótulos
Classifica um ponto novo pela votação dos vizinhos
Sem treino iterativo: guarda os dados

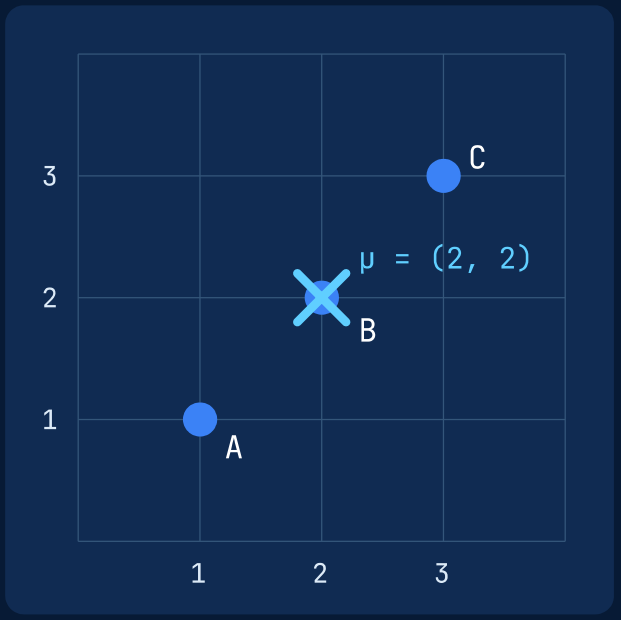


K-Means · K = quantidade de clusters

Não supervisionado · não usa rótulos
Agrupa todas as amostras em K grupos
Treino iterativo: move os centroides até estabilizar

BLOCO 4 · O QUE É UM CENTROIDE?

O centroide é a posição média do grupo



A = (1, 1) · B = (2, 2) · C = (3, 3)

Média em x $(1 + 2 + 3) / 3 = 2$

Média em y $(1 + 2 + 3) / 3 = 2$

Centroide $\mu = (2, 2)$

Aqui o centroide coincide com B por acaso. Com (0,0), (4,0) e (2,6), o centroide também é (2,2), e nenhuma amostra real está lá.

BLOCO 4 · FÓRMULA GERAL

Somar os pontos do grupo e dividir pela quantidade

$$\mu_k = \frac{1}{|C_k|} \sum_{x_i \in C_k} x_i$$

μ_k centroide do cluster *k*

C_k conjunto de amostras do cluster *k*

$|C_k|$ quantidade de amostras no cluster

Σ somatório: soma de todos os termos

x_i amostra *i* (vetor de características)

$\in$ «pertence a»

```
Exemplo: C = {A, B, C}, |C| = 3, soma = (6, 6) → μ = (6/3, 6/3) = (2, 2)
```

BLOCO 5 · FUNCIONAMENTO GERAL

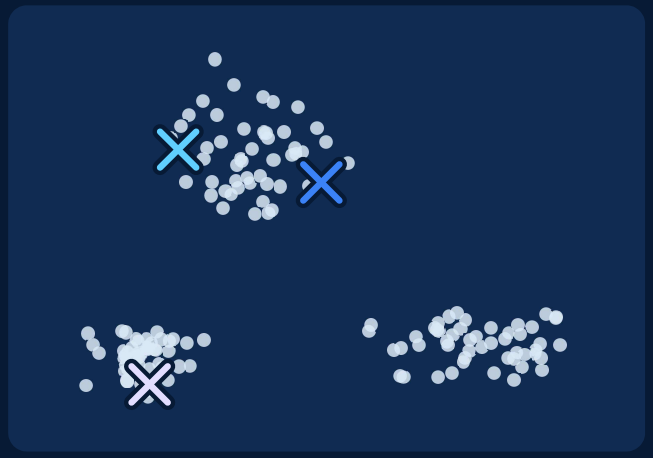
O algoritmo em sete etapas, ainda sem código

1	2	3	4	5	6	7
Definir K	Inicializar centroides	Calcular distâncias	Atribuir ao centroide mais próximo	Recalcular centroides	Verificar convergência	Repetir até o critério de parada

3 → 6 formam o laço iterativo: **atribuir** e **atualizar**, repetidamente. Cada volta é uma iteração.

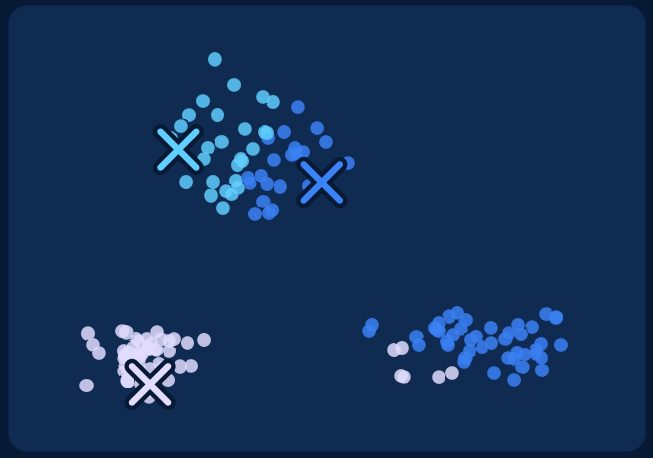
BLOCO 5 · SEQUÊNCIA VISUAL

Acompanhe os centroides se moverem



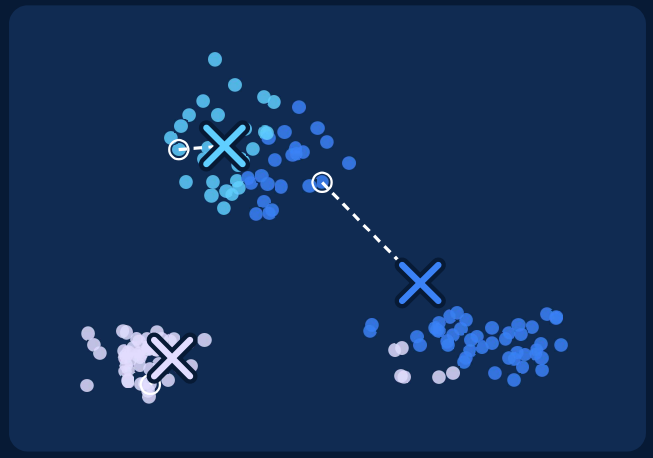
2 · Centroides iniciais

`3 amostras sorteadas`



3 · 1ª atribuição

`J = 223,830`



4 · 1ª atualização

`deslocamento máx. 1,253`

E depois? As atribuições mudam de novo, e os centroides voltam a se mover.

Eixo x: compras/mês · eixo y: gasto médio (padronizados) · X = centroide · círculo e tracejado = posição anterior e deslocamento.

BLOCO 6 · INICIALIZAÇÃO DOS CENTROIDES

Alguém precisa começar o ciclo

Inicialização simples `usada na aula`

Sortear K amostras do próprio dataset e copiá-las como centroides iniciais.

- Atribuir precisa de centroides; atualizar precisa de atribuições.
- O sorteio é controlado pela `random_state` (semente).
- Outra semente → outro ponto de partida → talvez outra solução final.

K-Means++ `conceito`

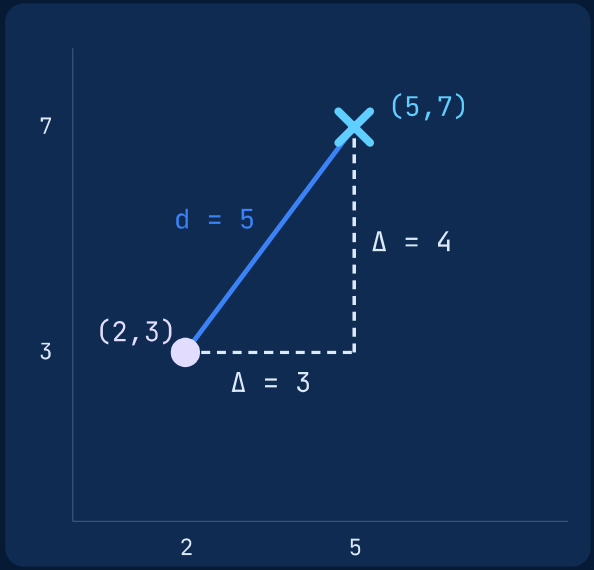
1. Primeiro centroide: uma amostra ao acaso.

2. Próximos: amostras sorteadas com probabilidade proporcional a $D(x)^2$, a distância ao centroide mais próximo já escolhido.

Finalidade: espalhar os centroides e reduzir soluções ruins. É o padrão do Scikit-learn.

BLOCO 7 · CÁLCULO DAS DISTÂNCIAS

Distância euclidiana: Pitágoras em *n* dimensões



$$d(x, \mu) = \sqrt{\left(\Sigma_j \, (x_j - \mu_j)^2 \right)}$$

d distância

x amostra · μ centroide

j índice da característica

$\sqrt{}$ raiz: volta à unidade original

$$d = \sqrt{((5-2)^2 + (7-3)^2)}$$

$$d = \sqrt{(9+16)} = \sqrt{25}$$

$$d = 5$$

BLOCO 7 · DISTÂNCIA COM NUMPY

Cada operação corresponde a um símbolo da fórmula

distancia.py

```
amostra = np.array([2, 3])
centroide = np.array([5, 7])
distancia = np.sqrt(np.sum((amostra - centroide) ** 2))
print(distancia)   # 5.0
```

`amostra - centroide`
diferença por coordenada
`[-3, -4]`

`** 2`
quadrado, elemento a elemento
`[9, 16]`

`np.sum(...)`
o somatório Σ
`25`

`np.sqrt(...)`
a raiz quadrada √
`5.0`

BLOCO 8 · ATRIBUIÇÃO DOS CLUSTERS

Cada amostra escolhe o centroide mais próximo

$\mu_0 = (1, 1)$ · $\mu_1 = (6, 5)$

amostra	$d(\mu_0)$	$d(\mu_1)$	label
(3, 2)	2,24	4,24	0
(2, 1)	1,00	5,66	0
(5, 4)	5,00	1,41	1

(3,2): $\sqrt{(2^2 + 1^2)} = \sqrt{5} \approx 2{,}24$ · $\sqrt{(3^2 + 3^2)} = \sqrt{18} \approx 4{,}24$

argmin.py

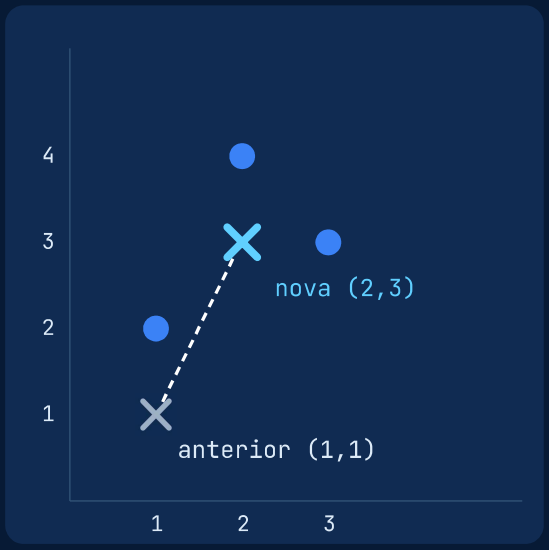
```
distancias = np.array([2.24, 4.24])
np.argmin(distancias)   # 0
```

`argmin` devolve a **posição** do menor valor, não o valor.

As distâncias estão na mesma ordem dos centroides; logo, a posição 0 é o centroide 0. O índice devolvido é o rótulo do cluster.

BLOCO 9 · ATUALIZAÇÃO DOS CENTROIDES

O centroide se move para a média do seu grupo



1 Separar os pontos do cluster: (1,2), (2,4), (3,3)

2 Média da 1ª característica: (1 + 2 + 3)/3 = 2

3 Média da 2ª característica: (2 + 4 + 3)/3 = 3

4 Atualizar: (1, 1) → (2, 3)

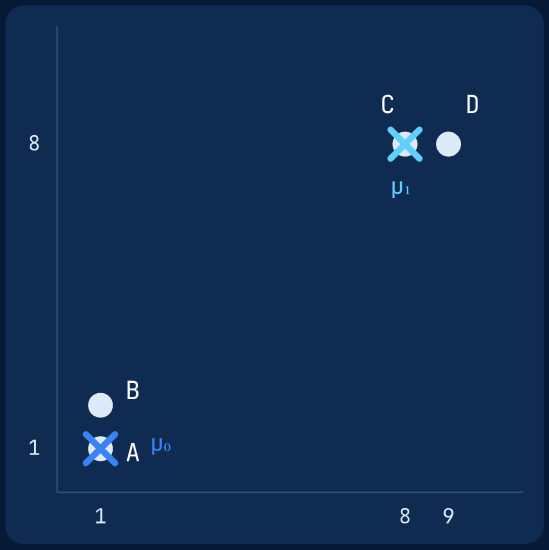
atualizar.py

```
pontos_cluster = np.array([[1, 2], [2, 4], [3, 3]])  # (3, 2)
novo_centroide = np.mean(pontos_cluster, axis=0)
print(novo_centroide)   # [2. 3.]  formato (2,)
```

`axis=0` percorre as linhas (amostras) e devolve **uma média por coluna**. A dimensão 0 desaparece.

BLOCO 10 · EXEMPLO COMPLETO · 1/4

Quatro pontos, K = 2: distâncias aos centroides

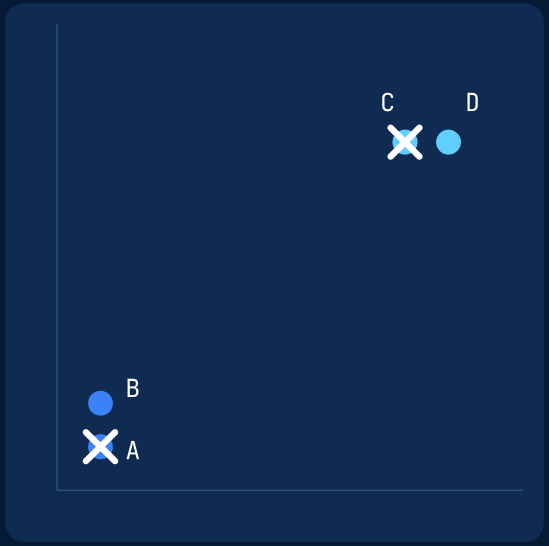


A(1,1) · B(1,2) · C(8,8) · D(9,8) · μ_0 = (1,1) · μ_1 = (8,8)

ponto	d(μ_0)	d(μ_1)
A	√0 = 0	√(49+49) = 9,90
B	√(0+1) = 1	√(49+36) = 9,22
C	√(49+49) = 9,90	√0 = 0
D	√(64+49) = 10,63	√(1+0) = 1

Exemplo: $d(D, \mu_0) = \sqrt{((9-1)^2 + (8-1)^2)} = \sqrt{(64 + 49)} = \sqrt{113} \approx 10{,}63$

Atribuição e contagem por cluster



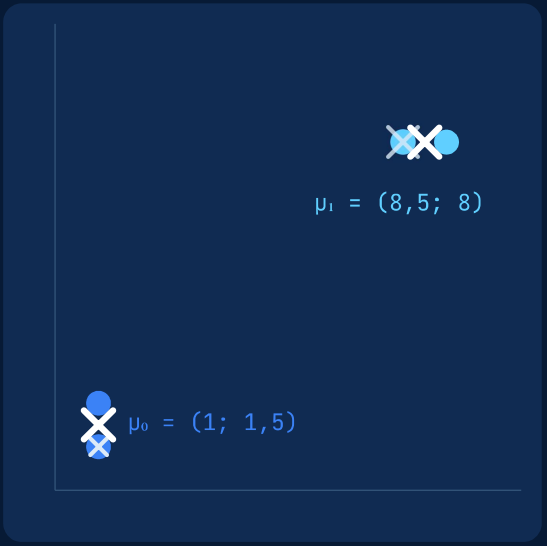
ponto	$d(\mu_0)$	$d(\mu_1)$	label
A	0	9,90	0
B	1	9,22	0
C	9,90	0	1
D	10,63	1	1

```
labels = [0, 0, 1, 1]
```

cluster 0 · {A, B} · 2 amostras cluster 1 · {C, D} · 2 amostras

A contagem é o divisor $|C_k|$ da média. Se fosse 0, a divisão produziria NaN.

Novos centroides: a média de cada grupo



Cluster 0 = {A(1,1), B(1,2)}

$$\bar{x} = (1 + 1)/2 = 1 \cdot \bar{y} = (1 + 2)/2 = 1{,}5$$

$$\mu_0 = (1; 1{,}5)$$

Cluster 1 = {C(8,8), D(9,8)}

$$\bar{x} = (8 + 9)/2 = 8{,}5 \cdot \bar{y} = (8 + 8)/2 = 8$$

$$\mu_1 = (8{,}5; 8)$$

$\bar{x}$, $\bar{y}$: médias de x e de y no cluster. Cada centroide se deslocou 0,5. Houve movimento: é preciso verificar as atribuições de novo.

Verificar atribuições, calcular inércia, decidir se parou

Reatribuição com os novos centroides

ponto	$d(\mu_0)$	$d(\mu_1)$	label
A	0,50	10,26	0
B	0,50	9,60	0
C	9,55	0,50	1
D	10,31	0,50	1

Nenhum label mudou → a próxima média é igual → deslocamento 0 → convergiu.

Inércia: soma das distâncias²

Centroides iniciais (1,1) e (8,8)

$$J = 0^2 + 1^2 + 0^2 + 1^2 = 2$$

Novos centroides (1; 1,5) e (8,5; 8)

$$J = 0{,}5^2 + 0{,}5^2 + 0{,}5^2 + 0{,}5^2 = 1$$

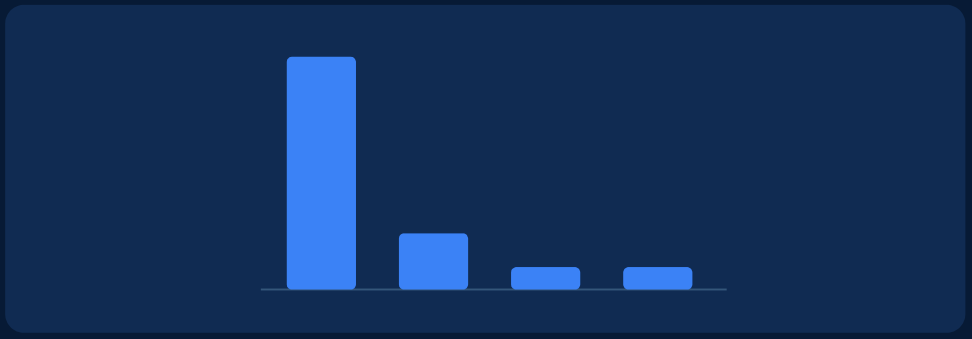
A inércia caiu de **2** para **1**. Trocar o centroide pela média nunca aumenta a inércia do grupo.

BLOCO 11 · ITERAÇÃO E CONVERGÊNCIA

Repetir até os centroides pararem

iteração	inércia J	deslocamento máx.
1	223,830	1,2530
2	54,205	0,7524
3	21,581	0,0000 ≤ tol

Dataset de clientes (150 × 2, padronizado), K = 3, random_state = 42.



Critérios de parada

1 · Centroides praticamente não mudam
maior deslocamento ≤ `tol` (tolerância, ex. 1e-4)

2 · Atribuições permanecem iguais
labels iguais geram as mesmas médias

3 · Número máximo de iterações
`max_iter`: limite de segurança, não um sinal de sucesso

Barras: inércia com os centroides C_0…C_3 do histórico.

BLOCO 12 · INÉRCIA

A quantidade que o K-Means tenta reduzir

$$J = \sum_{i=1}^{n} \| x_i - \mu_{c(i)} \|^2$$

J função objetivo, a inércia

Σ soma sobre as n amostras

x_i amostra i

$c(i)$ cluster atribuído à amostra i

$\mu_{c(i)}$ centroide desse cluster

$\| \|^2$ norma euclidiana ao quadrado

O quadrado elimina a raiz, penaliza mais as distâncias grandes e forma a **soma dos quadrados**, que a média minimiza.

BLOCO 12 · INÉRCIA COM NUMPY

centroides[labels] alinha cada amostra ao seu centroide

inercia.py

```
X = np.array([[1, 1], [1, 2], [8, 8], [9, 8]])   # (4, 2)
centroides = np.array([[1, 1.5], [8.5, 8]])       # (2, 2)
labels = np.array([0, 0, 1, 1])                   # (4,)
inercia = np.sum((X - centroides[labels]) ** 2)
print(inercia)   # 1.0
```

centroides[labels]
indexação: linhas μ_0, μ_0, μ_1, μ_1 → (4, 2)

X - ...
subtração linha a linha: $x_i - \mu_c(i)$

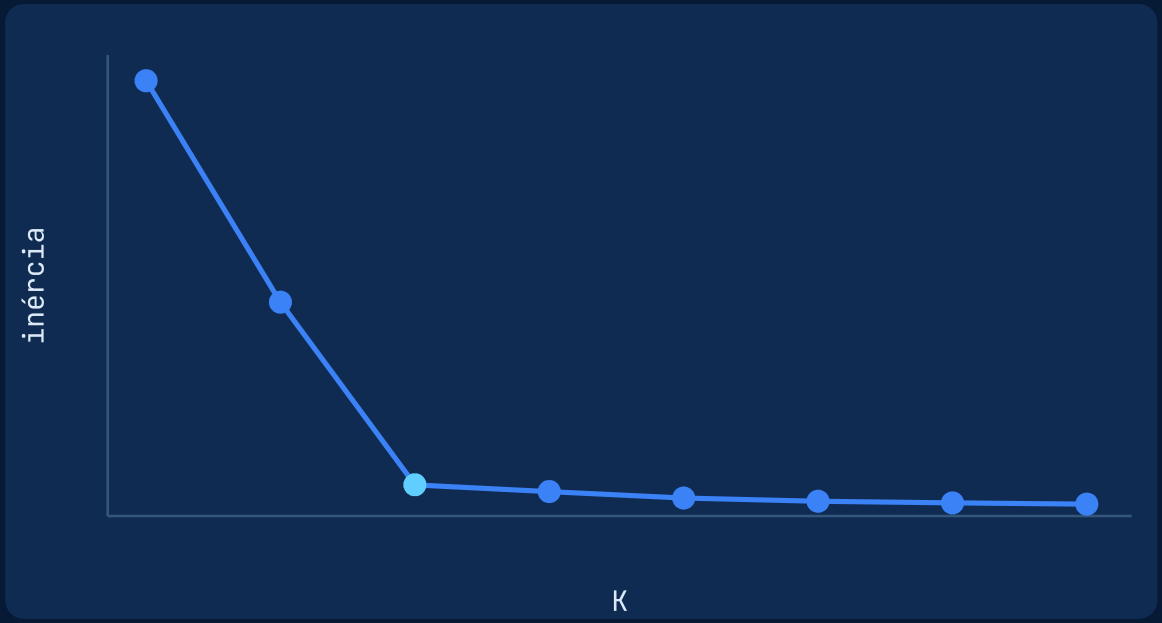
**** 2**
quadrado de cada coordenada

np.sum(...)
sem axis: soma tudo, coordenadas e amostras

Atenção: menor inércia = menor soma dos quadrados das distâncias. Não garante grupos com mais significado.

BLOCO 13 · MÉTODO DO COTOVELO

Como escolher K? Procure onde a queda perde força



Para cada K de 1 a 8: treinar, guardar a inércia.

Valores calculados pela nossa implementação (melhor de 10 sementes por K).

Queda forte até K = 3; depois, ganhos pequenos.

A inércia sempre diminui com K. Nem sempre existe cotovelo evidente: é uma heurística, não uma garantia.

BLOCO 13 · A CADEIA COMPLETA

K → agrupamento → inércia → cotovelo

1 · Escolher K

`n_clusters=k`

decisão nossa, antes do treino

2 · Agrupar

`fit(X) → labels_`

K centroides, um label por amostra

3 · Medir

`inercia_`

$J = \Sigma \|x_i - \mu_c(i)\|^2$

4 · Comparar

`K = 1, 2, …, 8`

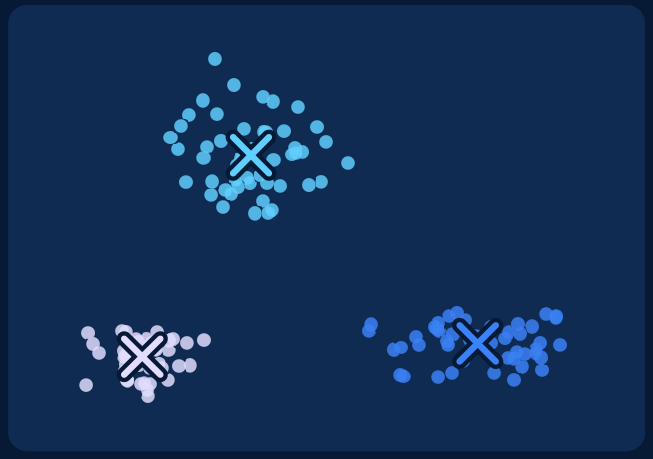
onde a queda perde força?

K	1	2	3	4	5
inércia	300,0	147,4	21,6	16,9	12,4
queda	–	-152,6	-125,8	-4,7	-4,5

Mais K nunca aumenta a melhor inércia; com K = n ela é 0. Por isso olhamos a mudança na intensidade da queda, não o menor valor.

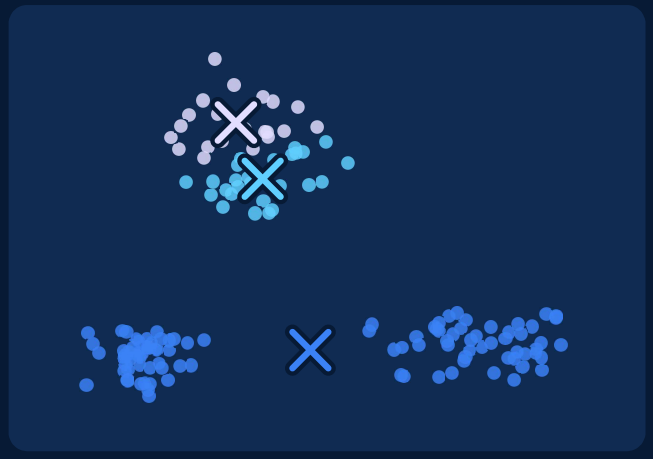
BLOCO 14 · INFLUÊNCIA DA SEMENTE

Mesmo dataset, mesmo K, três sementes



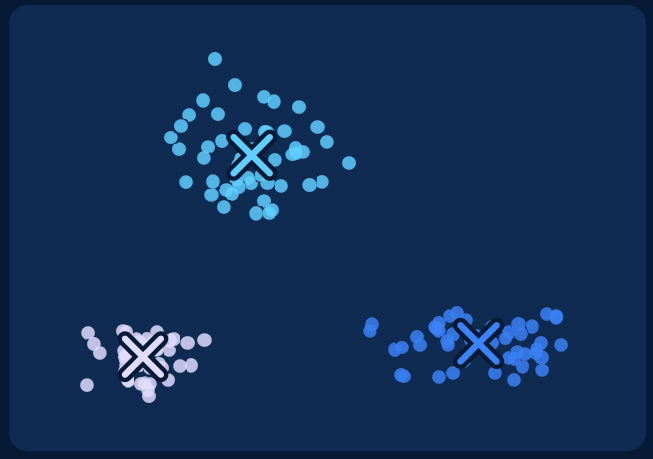
`random_state = 42`

`inércia 21,581 · 3 iterações`



`random_state = 1`

`inércia 150,566 · 9 iterações`



`random_state = 7`

`inércia 21,581 · 2 iterações`

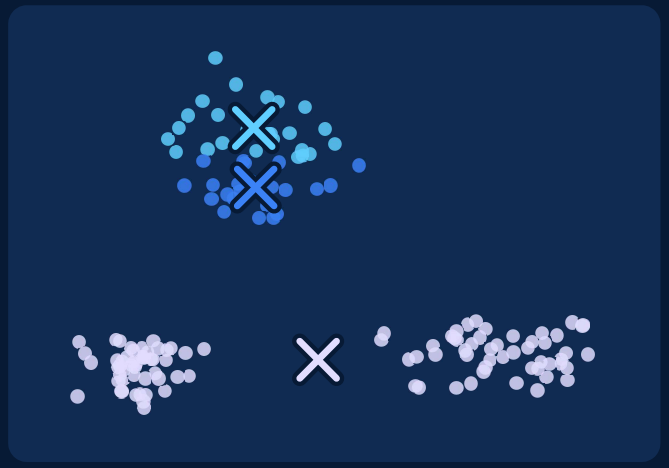
`random_state = 42` fixa o sorteio: mesma semente, mesmo resultado. Reprodutibilidade.

A semente 1 convergiu para um mínimo local. Solução: várias inicializações, ficar com a menor inércia.

Eixos padronizados (compras/mês × gasto médio) · cor = cluster · X = centroide final.

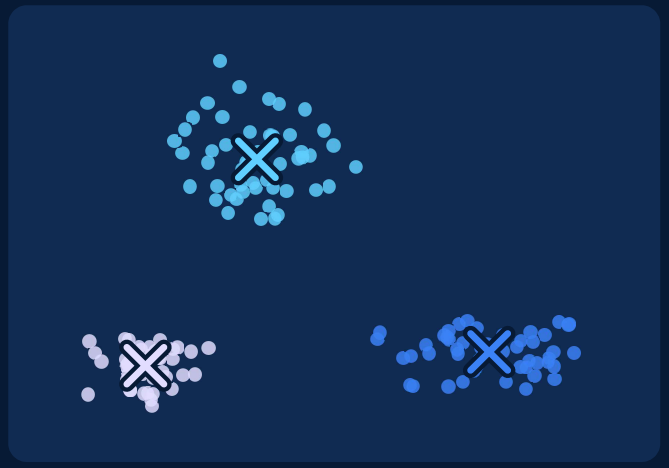
BLOCO 15 · ESCALONAMENTO

A característica de maior escala domina a distância



Antes: dados brutos

`grupos 25 / 25 / 100 · corta pelo gasto`



Depois: padronizado

`grupos 50 / 50 / 50`

`A(2 compras, R$60) · B(9, R$75) · C(2, R$80)`

$d(A,B) = \sqrt{(7^2 + 15^2)} = \mathbf{16{,}55}$

$d(A,C) = \sqrt{(0^2 + 20^2)} = \mathbf{20{,}00}$

A parece mais próximo de B, que compra 4× mais.

`StandardScaler`: (x − média) / desvio

`MinMaxScaler`: (x − mín) / (máx − mín)

Eixo x: compras por mês · eixo y: gasto médio por compra. Resultados reais, K = 3, random_state = 42.

BLOCO 16 · BROADCASTING COM NUMPY

Todas as distâncias de uma vez, sem laço

broadcasting.py

```
diferencas = X[:, None, :] - centroides[None, :, :]
distancias = np.sum(diferencas ** 2, axis=2)
labels = np.argmin(distancias, axis=1)
```

`X`	`(n, f)`	n amostras, f características
`centroides`	`(K, f)`	um centroide por linha
`X[:, None, :]`	`(n, 1, f)`	None cria um eixo de tamanho 1
`centroides[None, :, :]`	`(1, K, f)`	eixos de tamanho 1 são esticados
`diferencas`	`(n, K, f)`	cada amostra × cada centroide
`distancias (axis=2)`	`(n, K)`	soma das features: distância²
`labels (axis=1)`	`(n,)`	índice do menor valor de cada linha

BLOCO 16 · ARRAYS INTERMEDIÁRIOS

O exemplo de quatro pontos, dentro do NumPy

`X (4, 2)`

```
[[1, 1],
 [1, 2],
 [8, 8],
 [9, 8]]
```

`centroides (2, 2)`

```
[[1, 1],
 [8, 8]]
```

`diferencas (4, 2, 2)`

```
[[[ 0,  0], [-7, -7]],   A
 [[ 0,  1], [-7, -6]],   B
 [[ 7,  7], [ 0,  0]],   C
 [[ 8,  7], [ 1,  0]]]   D
```

bloco = amostra · linha = centroide · coluna = feature

`distancias (4, 2)`

```
[[  0,  98],
 [  1,  85],
 [ 98,   0],
 [113,   1]]
```

D: $8^2 + 7^2 = 113$ · $1^2 + 0^2 = 1$

`labels (4,)`

```
[0,
 0,
 1,
 1]
```

$\sqrt{98} = 9{,}90$ e $\sqrt{113} = 10{,}63$, como na tabela manual. A raiz é crescente: **a ordem das distâncias não muda**, e o centroide escolhido é o mesmo.

MAPA · MATEMÁTICA → CÓDIGO

Cada símbolo vira uma linha de NumPy

etapa	matemática	código
Inicialização	$\mu_k \leftarrow$ amostra sorteada	`X[np.argsort(uniformes(n, s))[:K]]`
Distância	$\|x_i - \mu_k\|^2$	`np.sum((X[:, None] - C[None]) ** 2, axis=2)`
Atribuição	$c(i) = \text{argmin}_k \|x_i - \mu_k\|^2$	`np.argmin(D, axis=1)`
Média	$(1/\|C_k\|) \, \Sigma_{x_i \in C_k} x_i$	`X[labels == k].mean(axis=0)`
Atualização	$\mu_k \leftarrow$ média do cluster k	`novos[k] = ... (vazio → amostra mais distante)`
Inércia	$J = \Sigma_i \|x_i - \mu_{c(i)}\|^2$	`D[np.arange(n), labels].sum()`
Convergência	$\max_k \|\mu_k^{\text{novo}} - \mu_k\| \leq \text{tol}$	`if desloc <= self.tol: break`

`inicializar → [ distância → atribuição → média → atualização ] → convergiu?`

GLOSSÁRIO · MATEMÁTICA

Glossário matemático do K-Means

K	quantidade de clusters, definida antes do treino
Cluster	amostras atribuídas ao mesmo centroide
Centroide μ_k	média das amostras do cluster k
Amostra x_i	uma linha de X (um cliente)
Característica	uma coluna de X (ex.: compras/mês)
Dist. euclidiana	√ da soma das diferenças²

Iteração	uma volta de atribuir + atualizar
Convergência	centroides param de se mover
Tolerância	deslocamento máximo tratado como zero
Inércia J	soma das distâncias² ao próprio centroide
$c(i)$	cluster atribuído à amostra i
$\| \| \cdot \Sigma \cdot \in$	norma (comprimento) · soma · pertence a

GLOSSÁRIO · CÓDIGO

Os nomes que aparecem no código

`labels`	cluster de cada amostra · (n,)
`centroides`	um centroide por linha · (K, f)
`max_iter`	limite de iterações (proteção)
`tol`	deslocamento máximo aceito como parada
`random_state`	semente do sorteio inicial

`axis`	dimensão percorrida; ela desaparece do resultado
`shape`	formato do array: tamanho de cada dimensão
`Broadcasting`	estica eixos de tamanho 1 para combinar arrays
`None`	em um índice, cria um eixo de tamanho 1
`atributo_`	sufixo _ : existe só depois do fit()

BLOCO 17 · IMPLEMENTAÇÃO DO ZERO

A classe KMeansDoZero

MÉTODOS

`__init__()`	guarda n_clusters, max_iter, tol, random_state
`fit(X)`	valida K e max_iter, executa as 7 etapas, devolve self
`predict(X)`	cluster mais próximo para amostras novas
`fit_predict(X)`	treina e devolve labels_

ATRIBUTOS (DEPOIS DO FIT)

`centroides_`	posições finais (K, f)
`labels_`	cluster de cada amostra (n,)
`inercia_`	inércia final
`n_iter_`	iterações executadas
`historico_inercia_`	J a cada iteração
`historico_centroides_`	posições a cada iteração

Somente Python e NumPy. Funções auxiliares: inicializar_centroides · calcular_distancias · atribuir_clusters · atualizar_centroides.

BLOCO 17 · DUAS REGRAS TÉCNICAS

Clusters vazios e estado final consistente

1 · Cluster vazio sem NaN

np.mean de um array vazio devolve NaN. Correção: mover o centroide vazio para a amostra mais distante do seu centroide atual.

• trecho de atualizar_centroides

```
if len(pontos) > 0:
    novos[k] = pontos.mean(axis=0)
else:
    mais_longe = np.argmax(d_atual)
    novos[k] = X[mais_longe]
    d_atual[mais_longe] = -1.0
```

2 · labels_ = centroides_ finais

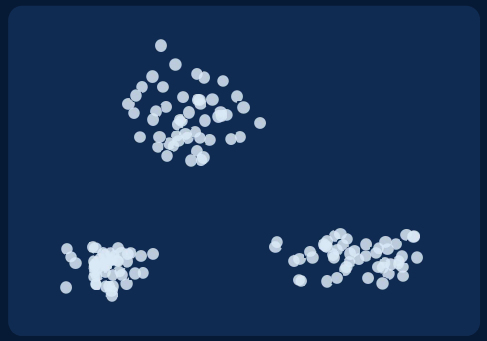
Os labels do laço foram calculados antes da última atualização. Depois do laço, recalcular tudo com os centroides finais.

• final de fit()

```
D = calcular_distancias(X, C)
self.centroides_ = C
self.labels_ = atribuir_clusters(D)
self.inercia_ = D[np.arange(len(X)),
                  self.labels_].sum()
```

BLOCO 18 · MOVIMENTAÇÃO DOS CENTROIDES

O histórico real do treinamento, em oito quadros



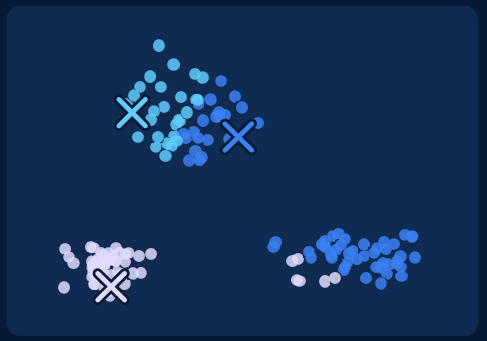
1 · Dataset original

`150 clientes, sem rótulos`



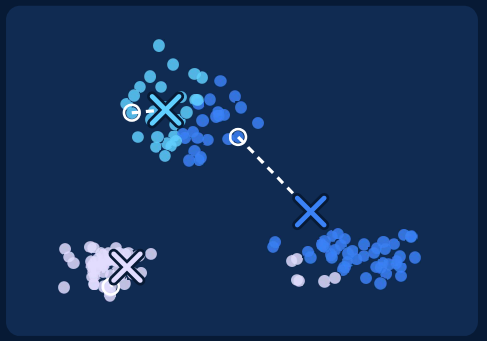
2 · Centroides iniciais

`3 amostras sorteadas`



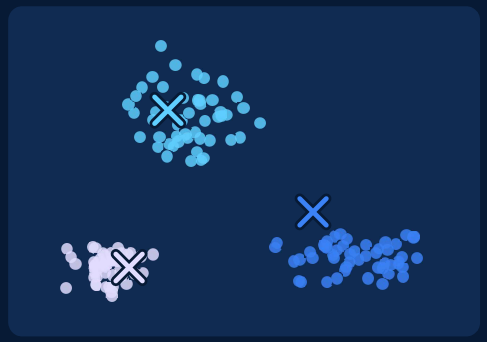
3 · 1ª atribuição

`J = 223,830`



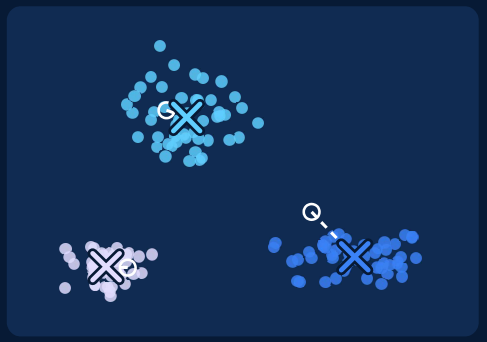
4 · 1ª atualização

`deslocamento máx. 1,253`



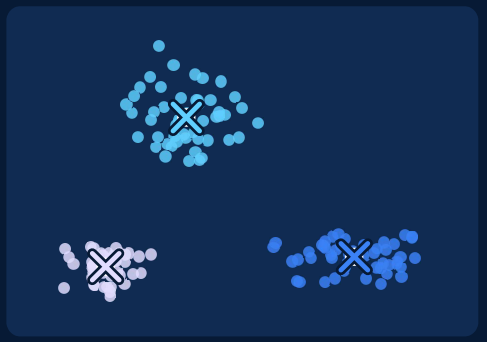
5 · Novas atribuições

`J = 54,205`



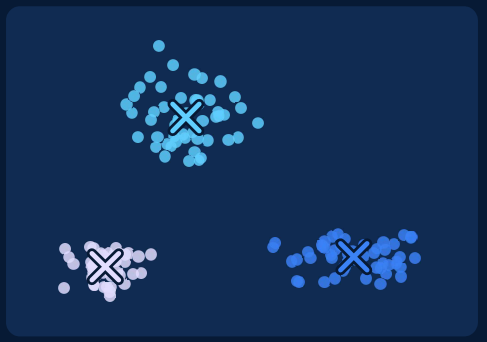
6 · Novas posições

`deslocamento máx. 0,752`



7 · Convergência

`deslocamento 0 ≤ tol`



8 · Resultado final

`J = 21,581 · n_iter_ = 3`

Eixos padronizados (compras/mês × gasto médio) · cor = cluster · X = centroide · círculo e tracejado = posição anterior e deslocamento · dados de historico_centroides_.

BLOCO 19 · EXERCÍCIO GUIADO

Segmentação de clientes, a partir de um notebook vazio

01 Imports
02 Criar o dataset
03 Visualizar os clientes
04 Preparar as características
05 Definir K
06 Inicializar centroides
07 Calcular as distâncias
08 Atribuir os clusters
09 Atualizar centroides
10 Laço iterativo
11 Verificar convergência
12 Criar a classe completa
13 Treinar o modelo
14 Visualizar os clusters
15 Calcular a inércia
16 Método do cotovelo
17 Comparar sementes
18 Comparar com Scikit-learn

Nenhuma variável, função ou dataset da teoria é reaproveitado.

Nada é usado antes de ser definido. A classe só é instanciada completa.

Restart Kernel → Run All deve executar sem erros.

EXERCÍCIO GUIADO · ETAPA 01 / 18

Imports

OBJETIVO

Carregar as duas bibliotecas da implementação.

POR QUÊ

NumPy faz as contas com arrays; Matplotlib desenha. O Scikit-learn só entra na etapa 18.

INTERPRETAÇÃO

Se a versão aparece, o ambiente está pronto. O número pode variar.

CONEXÃO

np será usado para gerar o dataset na etapa 02.

célula 1

```
import numpy as np
import matplotlib.pyplot as plt

print("NumPy", np.__version__)
```

SAÍDA

```
NumPy 1.26.4      # a versão pode variar
```

EXERCÍCIO GUIADO · ETAPA 02 / 18 · PARTE 1

Criar o dataset: um gerador reproduzível

OBJETIVO

Gerar ruído pseudoaleatório controlado por uma semente.

POR QUÊ

Mesma semente, mesmos números em qualquer máquina: os seus resultados batem com os dos slides.

INTERPRETAÇÃO

Soma de 3 uniformes − 1,5, vezes 2: média 0 e desvio 1.

CONEXÃO

uniformes também fará o sorteio dos centroides na etapa 06.

célula 2

```
def uniformes(n, semente):
    """n valores em [0, 1), sempre iguais para a mesma semente."""
    i = np.arange(1, n + 1)
    v = np.sin(i * 12.9898 + semente * 78.233) * 43758.5453
    return v - np.floor(v)          # parte fracionária

def ruido_normal(n, semente):
    """Aproximação da normal padrão (média 0, desvio 1)."""
    u = uniformes(3 * n, semente).reshape(3, n)
    return (u.sum(axis=0) - 1.5) * 2

print(uniformes(3, 0).round(3))
```

SAÍDA

```
[0.922 0.057 0.558]
```

EXERCÍCIO GUIADO · ETAPA 02 / 18 · PARTE 2

Criar o dataset: 150 clientes sintéticos

OBJETIVO

Matriz X com frequência de compras e gasto médio por compra.

POR QUÊ

Três perfis ocultos, 50 clientes cada. A origem de cada cliente não é guardada: X não tem rótulos.

INTERPRETAÇÃO

(150, 2): 150 amostras, 2 características. Coluna 0 = compras/mês; coluna 1 = R$.

CONEXÃO

Na etapa 03 olhamos esses pontos.

célula 3

```
centros = np.array([[2.0, 60.0], [9.0, 75.0], [4.5, 260.0]])
desvios = np.array([[0.6, 15.0], [1.0, 18.0], [0.9, 35.0]])
n = 50                                  # clientes por perfil

blocos = []
for g in range(3):
    s = 10 * g
    freq = centros[g, 0] + desvios[g, 0] * ruido_normal(n, s + 1)
    gasto = centros[g, 1] + desvios[g, 1] * ruido_normal(n, s + 2)
    blocos.append(np.column_stack([freq, gasto]))

X = np.vstack(blocos)                   # sem rótulos
print(X.shape)
print(X[:3].round(1))
```

SAÍDA

```
(150, 2)
[[ 1.7 38.9]
 [ 1.7 49.5]
 [ 2.4 55.6]]
```

EXERCÍCIO GUIADO · ETAPA 03 / 18

Visualizar os clientes

OBJETIVO

Gráfico de dispersão dos 150 clientes.

POR QUÊ

Olhar os dados antes de agrupar: densidade, escalas, possíveis grupos.

INTERPRETAÇÃO

Parecem existir 3 regiões. O eixo x vai até 12; o eixo y, até ~350.

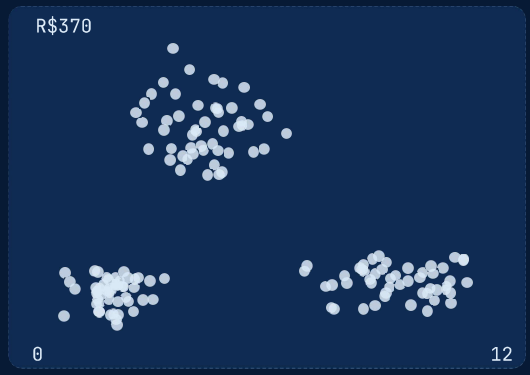
CONEXÃO

Escalas tão diferentes exigem preparação (etapa 04).

célula 4

```
plt.figure(figsize=(7, 5))
plt.scatter(X[:, 0], X[:, 1],
            s=20, color="gray")
plt.xlabel("Compras por mês")
plt.ylabel("Gasto médio (R$)")
plt.title("Clientes sem rótulos")
plt.show()
```

SAÍDA



EXERCÍCIO GUIADO · ETAPA 04 / 18

Analisar e preparar as características

OBJETIVO

Comparar escalas e padronizar as duas colunas.

POR QUÊ

O gasto varia ~30× mais que a frequência e dominaria a distância.

INTERPRETAÇÃO

X_esc tem média 0 e desvio 1 por coluna, como no StandardScaler. Coluna constante: desvio 0 vira 1 (sem NaN).

CONEXÃO

O K-Means trabalhará em X_esc. media e desvio voltam nas etapas 13 e 14.

célula 5

```
print("mínimo:", X.min(axis=0).round(1))
print("máximo:", X.max(axis=0).round(1))

media = X.mean(axis=0)                  # (2,)
desvio = X.std(axis=0)                  # (2,)
desvio = np.where(desvio == 0, 1.0, desvio)  # coluna constante
X_esc = (X - media) / desvio            # padronização

print("média:", media.round(3), "desvio:", desvio.round(3))
print("X_esc:", np.abs(X_esc.mean(axis=0)).round(3),
      X_esc.std(axis=0).round(3))
```

SAÍDA

```
mínimo: [ 0.8 24.2]
máximo: [ 11.1 347.6]
média: [  5.246 130.636] desvio: [ 3.153 91.391]
X_esc: [0. 0.] [1. 1.]
```

EXERCÍCIO GUIADO · ETAPA 05 / 18

Definir K

OBJETIVO

Escolher quantos grupos procurar.

POR QUÊ

O K-Means não descobre K. Usamos o gráfico (3 regiões) e a necessidade do negócio (poucos perfis acionáveis).

INTERPRETAÇÃO

K = 3 é uma hipótese inicial, não uma verdade.

CONEXÃO

K define quantos centroides sortear (etapa 06). O cotovelo (etapa 16) testa a hipótese.

célula 6

```
K = 3
print("Vamos procurar", K, "grupos de clientes.")
```

SAÍDA

```
Vamos procurar 3 grupos de clientes.
```

EXERCÍCIO GUIADO · ETAPA 06 / 18

Inicializar os centroides

OBJETIVO

Sortear K amostras distintas como centroides iniciais.

POR QUÊ

argsort de números aleatórios é uma permutação: os K primeiros índices são um sorteio sem repetição.

INTERPRETAÇÃO

(3, 2): um centroide por linha. Dois caíram no grupo de ticket alto.

CONEXÃO

Medimos a distância de cada cliente a eles (etapa 07).

célula 7

```
def inicializar_centroides(X, K, semente):
    """Sorteia K amostras distintas de X."""
    indices = np.argsort(uniformes(len(X), semente))[:K]
    return X[indices].copy()

centroides = inicializar_centroides(X_esc, K, semente=42)
print(np.argsort(uniformes(len(X_esc), 42))[:K])
print(centroides.round(3))
```

SAÍDA

```
[112 109  41]
[[ 0.213  1.082]
 [-0.772  1.425]
 [-0.967 -1.039]]
```

EXERCÍCIO GUIADO · ETAPA 07 / 18

Calcular as distâncias

OBJETIVO

Matriz (n, K) com a distância² de cada cliente a cada centroide.

POR QUÊ

Broadcasting calcula as 450 distâncias sem laço. Sem raiz: a ordem não muda.

INTERPRETAÇÃO

Linha = cliente, coluna = centroide. O cliente 0 está a 0,025 do centroide 2.

CONEXÃO

O menor valor de cada linha define o cluster (etapa 08).

célula 8

```
def calcular_distancias(X, centroides):
    """Distâncias AO QUADRADO amostra × centroide: (n, K)."""
    diferencas = X[:, None, :] - centroides[None, :, :]  # (n,K,f)
    return np.sum(diferencas ** 2, axis=2)               # (n,K)

D = calcular_distancias(X_esc, centroides)
print(D.shape)
print(D[:3].round(3))
```

SAÍDA

```
(150, 3)
[[6.129 6.023 0.025]
 [5.648 5.47  0.045]
 [4.872 5.065 0.051]]
```

EXERCÍCIO GUIADO · ETAPA 08 / 18

Atribuir os clusters

OBJETIVO

Cada cliente recebe o índice do centroide mais próximo.

POR QUÊ

argmin(axis=1) percorre os K centroides de cada linha. Argmax escolheria o mais distante.

INTERPRETAÇÃO

66 / 28 / 56: desequilibrado, porque a inicialização foi ruim. Ainda não é o resultado.

CONEXÃO

Com os labels, recalculamos as médias (etapa 09).

célula 9

```
def atribuir_clusters(distancias):
    """Índice do centroide mais próximo de cada amostra: (n,)."""
    return np.argmin(distancias, axis=1)

labels = atribuir_clusters(D)
print(labels[:3])
print(np.bincount(labels, minlength=K))
```

SAÍDA

```
[2 2 2]
[66 28 56]
```

EXERCÍCIO GUIADO · ETAPA 09 / 18

Atualizar os centroides

OBJETIVO

Mover cada centroide para a média do seu cluster.

POR QUÊ

A média minimiza a soma dos quadrados. Cluster vazio daria NaN: recebe a amostra mais distante.

INTERPRETAÇÃO

O centroide 0 desceu de (0,21; 1,08) para (0,89; 0,03).

CONEXÃO

Testamos o cluster vazio antes de seguir.

• célula 10

```
def atualizar_centroides(X, labels, centroides, distancias):
    """Média de cada cluster; vazio recebe a amostra mais distante."""
    novos = centroides.copy()
    d_atual = distancias[np.arange(len(X)), labels].copy()
    for k in range(len(centroides)):
        pontos = X[labels == k]
        if len(pontos) > 0:
            novos[k] = pontos.mean(axis=0)
        else:
            mais_longe = np.argmax(d_atual)
            novos[k] = X[mais_longe]
            d_atual[mais_longe] = -1.0
    return novos

novos = atualizar_centroides(X_esc, labels, centroides, D)
print(novos.round(3))
```

```
[[ 0.888  0.026]
 [-0.459  1.464]
 [-0.818 -0.763]]
```

EXERCÍCIO GUIADO · ETAPA 09 / 18 · TESTE

Testar o cluster vazio

OBJETIVO

Provar que um cluster vazio não gera NaN.

POR QUÊ

O centroide em (50,50) não recebe ninguém. Números pequenos permitem conferir de cabeça.

INTERPRETAÇÃO

Contagem [2 2 0]. O centroide 2 vai para a amostra (0, 0). Nenhum NaN.

CONEXÃO

Com as 4 funções validadas, montamos o laço (etapa 10).

célula 11

```
X_teste = np.array([[0, 0], [0, 1], [10, 10], [10, 11]], dtype=float)
C_teste = np.array([[0, 0.5], [10, 10.5], [50, 50]])

D_teste = calcular_distancias(X_teste, C_teste)
L_teste = atribuir_clusters(D_teste)
print(L_teste, np.bincount(L_teste, minlength=3))

C_novo = atualizar_centroides(X_teste, L_teste, C_teste, D_teste)
print(C_novo)
print("tem NaN?", np.isnan(C_novo).any())
```

SAÍDA

```
[0 0 1 1] [2 2 0]
[[ 0.   0.5]
 [10.  10.5]
 [ 0.   0. ]]
tem NaN? False
```

EXERCÍCIO GUIADO · ETAPA 10 / 18

Implementar o laço iterativo

OBJETIVO

Repetir atribuição e atualização e observar a inércia.

POR QUÊ

Uma volta não basta: novos centroides mudam atribuições, que mudam médias.

INTERPRETAÇÃO

A inércia nunca sobe e estabiliza em 21,581 na 3ª iteração.

CONEXÃO

As voltas 4 e 5 foram inúteis: falta parar sozinho (etapa 11).

célula 12

```
C = inicializar_centroides(X_esc, K, semente=42)

for it in range(1, 6):
    D = calcular_distancias(X_esc, C)
    labels = atribuir_clusters(D)
    J = D[np.arange(len(X_esc)), labels].sum()   # inércia
    print(f"iteração {it}: inércia = {J:.3f}")
    C = atualizar_centroides(X_esc, labels, C, D)
```

SAÍDA

```
iteração 1: inércia = 223.830
iteração 2: inércia = 54.205
iteração 3: inércia = 21.581
iteração 4: inércia = 21.581
iteração 5: inércia = 21.581
```

EXERCÍCIO GUIADO · ETAPA 11 / 18

Verificar a convergência

OBJETIVO

Parar quando os centroides deixam de se mover.

POR QUÊ

tol define «parado»; max_iter garante que o laço termina.

INTERPRETAÇÃO

Deslocamentos 1,253 → 0,752 → 0: convergiu na 3ª iteração.

CONEXÃO

Este laço vira o método fit da classe (etapa 12).

célula 13

```
tol = 1e-4
max_iter = 100
C = inicializar_centroides(X_esc, K, semente=42)

for it in range(1, max_iter + 1):
    D = calcular_distancias(X_esc, C)
    labels = atribuir_clusters(D)
    novos = atualizar_centroides(X_esc, labels, C, D)
    desloc = np.sqrt(((novos - C) ** 2).sum(axis=1)).max()
    print(f"iteração {it}: deslocamento máximo = {desloc:.4f}")
    C = novos
    if desloc ⩽ tol:
        print("convergiu")
        break
```

```
iteração 1: deslocamento máximo = 1.2530
iteração 2: deslocamento máximo = 0.7524
iteração 3: deslocamento máximo = 0.0000
convergiu
```

EXERCÍCIO GUIADO · ETAPA 12 / 18 · CÉLULA 14 · PARTE 1 DE 3

A classe completa: construtor e início do fit

célula 14 · linhas 1–18

```
1  class KMeansDoZero:
2      def __init__(self, n_clusters=3, max_iter=100,
3                   tol=1e-4, random_state=0):
4          self.n_clusters = n_clusters
5          self.max_iter = max_iter
6          self.tol = tol
7          self.random_state = random_state
8
9      def fit(self, X):
10         X = np.asarray(X, dtype=float)
11         if not 1 ⩽ self.n_clusters ⩽ len(X):
12             raise ValueError("n_clusters deve estar entre 1 e o número de amostras")
13         if self.max_iter < 1:
14             raise ValueError("max_iter deve ser ⩾ 1")
15         C = inicializar_centroides(X, self.n_clusters,
16                                    self.random_state)
17         self.historico_centroides_ = [C.copy()]
18         self.historico_inercia_ = []
```

1–7 guardam os hiperparâmetros; nada é treinado.

10 garante array de floats.

11–14 validam K (1 ≤ K ≤ n) e max_iter ≥ 1.

15–16 etapa 06.

17–18 históricos com .copy().

Continua na mesma célula →

EXERCÍCIO GUIADO · ETAPA 12 / 18 · CÉLULA 14 · PARTE 2 DE 3

A classe completa: o laço de treinamento

célula 14 · linhas 19-29

```
19          for it in range(1, self.max_iter + 1):
20              D = calcular_distancias(X, C)
21              labels = atribuir_clusters(D)
22              J = D[np.arange(len(X)), labels].sum()
23              self.historico_inercia_.append(J)
24              novos = atualizar_centroides(X, labels, C, D)
25              desloc = np.sqrt(((novos - C) ** 2).sum(axis=1)).max()
26              C = novos
27              self.historico_centroides_.append(C.copy())
28              if desloc ⩽ self.tol:
29                  break
```

20–21 etapas 07 e 08.

22–23 inércia com os centroides atuais.

24 etapa 09, com cluster vazio tratado.

25–29 etapa 11: deslocamento e parada.

Continua na mesma célula →

EXERCÍCIO GUIADO · ETAPA 12 / 18 · CÉLULA 14 · PARTE 3 DE 3

A classe completa: estado final, predict e fit_predict

célula 14 · linhas 30-47

```
30          # estado final: tudo com os centroides FINAIS
31          D = calcular_distancias(X, C)
32          self.centroides_ = C
33          self.labels_ = atribuir_clusters(D)
34          self.inercia_ = D[np.arange(len(X)), self.labels_].sum()
35          self.historico_inercia_.append(self.inercia_)
36          self.n_iter_ = it
37          return self
38
39      def predict(self, X):
40          if not hasattr(self, "centroides_"):
41              raise RuntimeError("chame fit() antes de predict()")
42          D = calcular_distancias(np.asarray(X, dtype=float),
43                                  self.centroides_)
44          return atribuir_clusters(D)
45
46      def fit_predict(self, X):
47          return self.fit(X).labels_
```

31–34 labels_ e inercia_ com os centroides_ finais.

35–37 histórico completo, n_iter_, devolve self.

39–44 predict exige fit e não move centroides.

A célula não imprime nada: apenas define a classe.

EXERCÍCIO GUIADO · ETAPA 13 / 18

Treinar o modelo e testar fit, predict e fit_predict

OBJETIVO

Treinar e validar os três métodos.

POR QUÊ

O assert prova a consistência. Clientes novos são padronizados com media e desvio do treino.

INTERPRETAÇÃO

3 iterações, 50/50/50, mesmo histórico do laço manual.

CONEXÃO

Agora desenhamos os grupos (etapa 14).

célula 15

```
modelo = KMeansDoZero(n_clusters=3, random_state=42)
modelo.fit(X_esc)
print("iterações:", modelo.n_iter_, "| inércia:", round(modelo.inercia_, 3))
print("clientes por cluster:", np.bincount(modelo.labels_))
print("histórico:", np.round(modelo.historico_inercia_, 3))

D_final = calcular_distancias(X_esc, modelo.centroides_)
assert np.array_equal(modelo.labels_, atribuir_clusters(D_final))

novos_clientes = np.array([[6, 200], [1.5, 50], [10, 90]])
print(modelo.predict((novos_clientes - media) / desvio))
outro = KMeansDoZero(3, random_state=42)
print(np.array_equal(outro.fit_predict(X_esc), modelo.labels_))
```

```
iterações: 3 | inércia: 21.581
clientes por cluster: [50 50 50]
histórico: [223.83   54.205  21.581  21.581]
[1 2 0]
True
```

EXERCÍCIO GUIADO · ETAPA 13 / 18 · TESTES DE ROBUSTEZ

Testar os casos difíceis

O QUE ESPERAR

K = 1 centroide = média; J = n × f = 300

K > n ValueError com mensagem clara

Repetidos cluster vazio reposicionado; sem NaN; J = 0

max_iter = 1 para após 1 iteração; labels_ coerentes

Histórico cópias, não referências

Coluna constante desvio 0 → 1 na etapa 04

• célula 16

```
m1 = KMeansDoZero(1).fit(X_esc)                                  # K = 1
print("K=1:", np.allclose(m1.centroides_, X_esc.mean(axis=0)),
      round(m1.inercia_, 3))
try:                                                             # K > n
    KMeansDoZero(5).fit(X_esc[:3])
except ValueError as erro:
    print("K>n:", erro)
X_rep = np.array([[0, 0]] * 3 + [[5, 5]] * 3, dtype=float)   # repetidos
m_rep = KMeansDoZero(3, random_state=0).fit(X_rep)
print("repetidos:", np.isnan(m_rep.centroides_).any(), round(m_rep.inercia_, 3))
m_lim = KMeansDoZero(3, max_iter=1, random_state=42).fit(X_esc)
print("max_iter=1:", m_lim.n_iter_, round(m_lim.inercia_, 3))
H = modelo.historico_centroides_                                 # cópias
print("cópias:", not np.shares_memory(H[0], H[-1]))
```

```
K=1: True 300.0
K>n: n_clusters deve estar entre 1 e o número de amostras
repetidos: False 0.0
max_iter=1: 1 54.205
cópias: True
```

EXERCÍCIO GUIADO · ETAPA 14 / 18

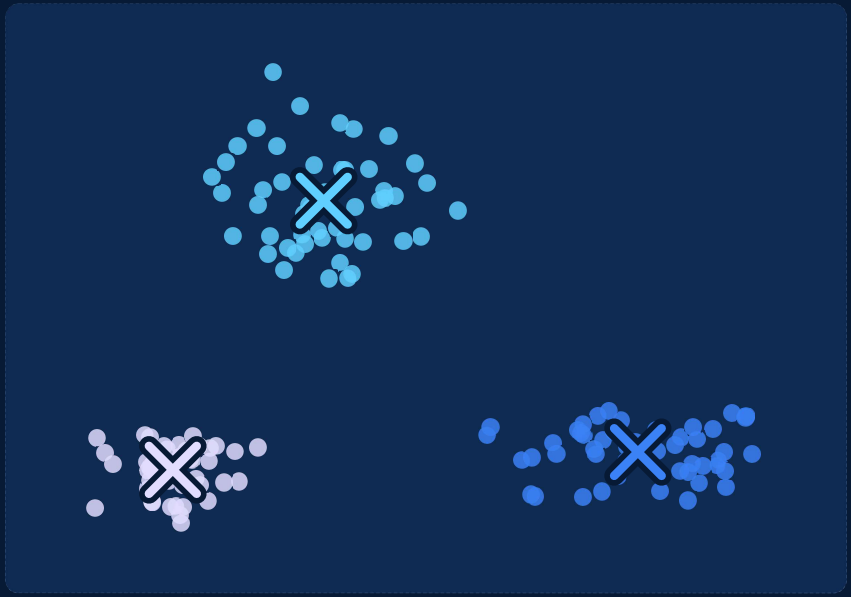
Visualizar os clusters nas unidades originais

• célula 17

```
cores = np.array(["#3B82F6", "#60CFFF", "#E2DDFF"])
C_orig = modelo.centroides_ * desvio + media  # desfaz

plt.figure(figsize=(7, 5))
plt.scatter(X[:, 0], X[:, 1],
            c=cores[modelo.labels_], s=20)
plt.scatter(C_orig[:, 0], C_orig[:, 1], marker="X",
            s=250, c=cores, edgecolor="black")
plt.xlabel("Compras por mês")
plt.ylabel("Gasto médio (R$)")
plt.show()
print(C_orig.round(1))
```

```
[[  9.3  74.8]
 [  4.4 255.2]
 [  2.   61.9]]
```



INTERPRETAÇÃO

Três grupos separados; cada X no centro do seu grupo.

CONEXÃO

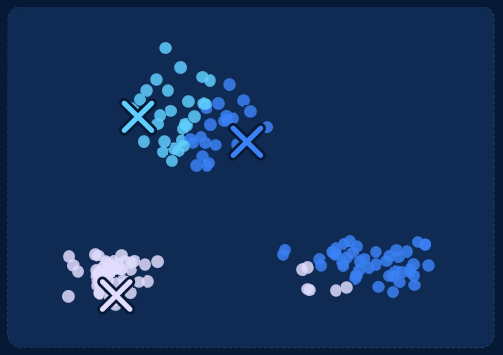
A seguir: o caminho dos centroides até aqui.

EXERCÍCIO GUIADO · ETAPA 14 / 18 · HISTÓRICO

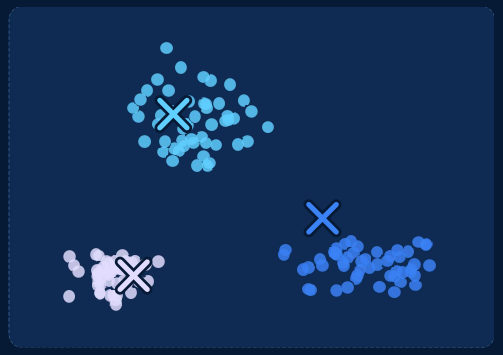
Desenhar o caminho real dos centroides

• célula 18

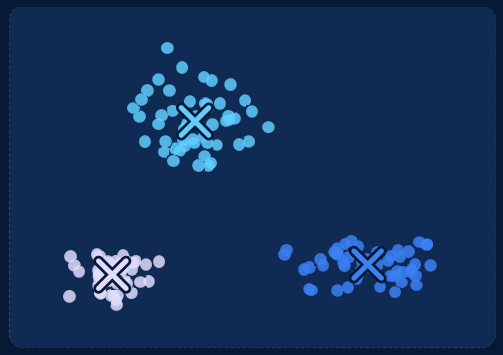
```
H = modelo.historico_centroides_                    # C0, C1, C2, C3
fig, eixos = plt.subplots(1, len(H), figsize=(4 * len(H), 3.5))
for t, (ax, C_t) in enumerate(zip(eixos, H)):
    rot = atribuir_clusters(calcular_distancias(X_esc, C_t))
    ax.scatter(X_esc[:, 0], X_esc[:, 1], c=cores[rot], s=10)
    ax.scatter(C_t[:, 0], C_t[:, 1], marker="X", s=200, c=cores, edgecolor="black")
    ax.set_title(f"centroides C{t}")
plt.show()
```



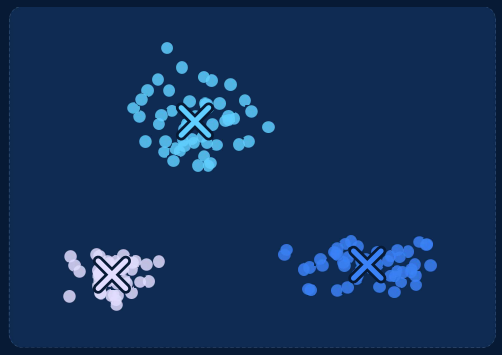
centroides C0



centroides C1



centroides C2



centroides C3

EXERCÍCIO GUIADO · ETAPA 15 / 18

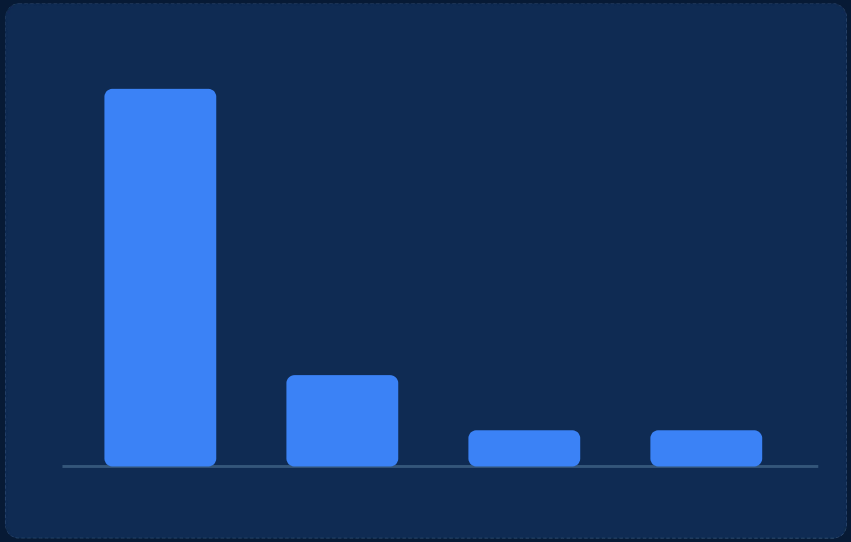
Calcular a inércia de duas formas

célula 19

```
J1 = modelo.inercia_
J2 = np.sum((X_esc - modelo.centroides_[modelo.labels_]) ** 2)
print(round(J1, 3), round(J2, 3), np.isclose(J1, J2))

h = modelo.historico_inercia_
plt.plot(range(len(h)), h, marker="o")
plt.xlabel("centroides C_t")
plt.ylabel("inércia")
plt.show()
```

```
21.581 21.581 True
```



INTERPRETAÇÃO

As duas fórmulas concordam. A curva só desce: se subisse, haveria um bug.

CONEXÃO

E se K fosse outro? Etapa 16.

EXERCÍCIO GUIADO · ETAPA 16 / 18

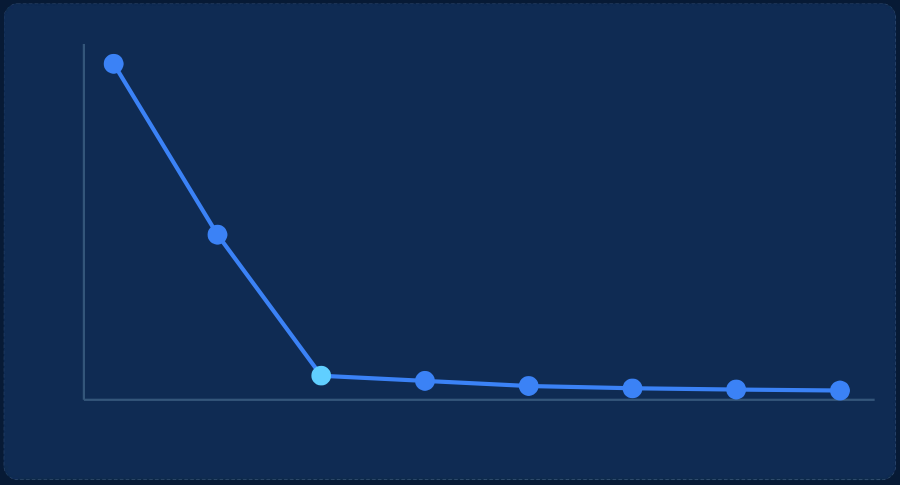
Método do cotovelo

• célula 20

```
valores_K = range(1, 9)
inercias = []
for k in valores_K:
    melhor = min(KMeansDoZero(k, random_state=s)
                 .fit(X_esc).inercia_ for s in range(10))
    inercias.append(melhor)
print(np.round(inercias, 1))

plt.plot(valores_K, inercias, marker="o")
plt.xlabel("K")
plt.ylabel("inércia")
plt.show()
```

```
[300.  147.4  21.6  16.9  12.4  10.2   9.1   8.3]
```



INTERPRETAÇÃO

Queda de 147,4 para 21,6 e depois ganhos pequenos: cotovelo em K = 3.

CONEXÃO

Por que usamos 10 sementes? Etapa 17.

EXERCÍCIO GUIADO · ETAPA 17 / 18

Comparar sementes

OBJETIVO

Ver o efeito da inicialização no resultado final.

POR QUÊ

O K-Means só melhora localmente: o ponto de partida importa.

INTERPRETAÇÃO

Semente 1 convergiu para um mínimo local (150,566). A melhor de 10 recupera 21,581.

CONEXÃO

O n_init do Scikit-learn faz isso (etapa 18).

• célula 21

```
for semente in [42, 1, 7]:
    m = KMeansDoZero(3, random_state=semente).fit(X_esc)
    print(f"semente {semente:2d}: inércia = {m.inercia_:7.3f} | "
          f"iterações = {m.n_iter_} | "
          f"tamanhos = {np.bincount(m.labels_, minlength=3)}")

modelos = [KMeansDoZero(3, random_state=s).fit(X_esc) for s in range(10)]
melhor = min(modelos, key=lambda m: m.inercia_)
print("melhor inércia entre 10 sementes:", round(melhor.inercia_, 3))
```

SAÍDA

```
semente 42: inércia =  21.581 | iterações = 3 | tamanhos = [50 50 50]
semente  1: inércia = 150.566 | iterações = 9 | tamanhos = [100  29  21]
semente  7: inércia =  21.581 | iterações = 2 | tamanhos = [50 50 50]
melhor inércia entre 10 sementes: 21.581
```

EXERCÍCIO GUIADO · ETAPA 18 / 18

Comparar com o Scikit-learn

OBJETIVO

Conferir nossa implementação com a da biblioteca.

POR QUÊ

A numeração dos clusters pode mudar: pareamos centroides antes de comparar labels.

INTERPRETAÇÃO

Mesma inércia, mesmos centroides, 100% de concordância após o pareamento.

CONEXÃO

Kernel → Restart & Run All.

célula 22

```
from sklearn.cluster import KMeans

sk = KMeans(n_clusters=3, n_init=10, random_state=42).fit(X_esc)
print("inércia sklearn:", round(sk.inertia_, 3))
print("inércia nossa:  ", round(modelo.inercia_, 3))

# par[k] = centroide do sklearn mais próximo do nosso centroide k
par = atribuir_clusters(
    calcular_distancias(modelo.centroides_, sk.cluster_centers_))
print("pareamento:", par)
dif = np.abs(modelo.centroides_ - sk.cluster_centers_[par]).max()
print("maior diferença:", round(dif, 6))
print("concordância:", (par[modelo.labels_] == sk.labels_).mean())
```

SAÍDA ESPERADA · a ordem de «pareamento» pode variar

```
inércia sklearn: 21.581
inércia nossa:   21.581
pareamento: [2 0 1]
maior diferença: 0.0
concordância: 1.0
```

BLOCO 21 · INTERPRETAÇÃO DOS CLUSTERS

O significado vem dos dados, não do número do cluster

cluster	clientes	compras/mês	gasto/compra	leitura
0	50	9,3	R$ 74,8	frequentes, ticket baixo
1	50	4,4	R$ 255,2	ticket alto
2	50	2,0	R$ 61,9	ocasionais, ticket baixo

Cluster 0 não é «baixo gasto». O menor gasto está no cluster 2.

Os identificadores são índices; outra semente pode renumerar os mesmos grupos.

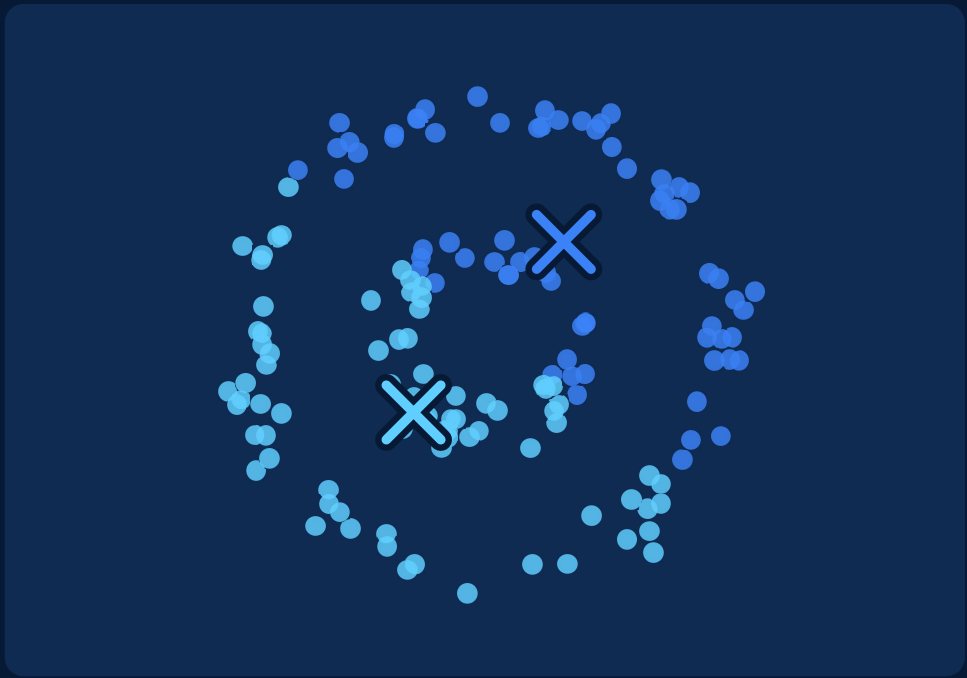
Inércia final 21,58: grupos compactos. Utilidade é decisão de negócio.

```
for k in range(3): print(k, X[modelo.labels_ == k].mean(axis=0).round(1))
```

BLOCO 22 · LIMITAÇÕES DO K-MEANS

Quando o K-Means não é a ferramenta certa

Inicialização	semente 1: mínimo local, J = 150,6
Escolha de K	nem sempre há cotovelo
Escalas	sem padronizar: 25 / 25 / 100
Outliers	a média é puxada por valores extremos
Formatos irregulares	fronteiras são retas entre centroides
Mínimos locais	convergir não garante a melhor solução
Memória	diferenças ocupam n × K × f números



Dois anéis concêntricos, K = 2: o K-Means corta os anéis ao meio em vez de separá-los.

Alternativas para formatos irregulares: DBSCAN, clusterização espectral.

BLOCO 23 · COMPARAÇÃO COM SCIKIT-LEARN

A biblioteca faz o que acabamos de construir

sklearn.py

```
from sklearn.cluster import KMeans

sk = KMeans(n_clusters=3, n_init=10,
            random_state=42)
sk.fit(X_esc)
sk.predict((novos_clientes - media) / desvio)
```

Scikit-learn	KMeansDoZero
n_clusters	n_clusters
cluster_centers_	centroides_
labels_	labels_
inertia_	inercia_

`n_init`: quantas inicializações; fica com a menor inércia.

`random_state`: semente; torna o resultado reprodutível.

`fit()` treina · `predict()` atribui amostras novas.

Comparar: centroides, inércia, nº de clusters, distribuição dos pontos, gráfico.

Não exija labels idênticos. O mesmo agrupamento pode vir numerado como [2 0 1]. Pareie os centroides antes.

BLOCO 24 · QUIZ · 1/2

Responda sem olhar o código

01 O que é aprendizado não supervisionado?

02 Qual a diferença entre KNN e K-Means?

03 O que representa K?

04 O que é um centroide?

05 Como uma amostra escolhe seu cluster?

06 Como atualizamos os centroides?

07 O que significa convergência?

BLOCO 24 · QUIZ · 2/2

Mais sete perguntas

08 O que é inércia?

09 Para que serve o método do cotovelo?

10 O que representa `random_state`?

11 Por que escalonamos as características?

12 O que faz `np.argmin()`?

13 O que significa `axis=0`?

14 Por que dois treinamentos podem produzir agrupamentos diferentes?

Causa e correção

erro	causa	correção
Confundir K-Means com KNN	os dois usam K e distância	KNN: K vizinhos, rótulos. K-Means: K grupos
`np.argmax()`	escolhe o centroide mais distante	np.argmin(distancias, axis=1)
Distâncias incorretas	sem quadrado ou soma no eixo errado	np.sum(dif ** 2, axis=2)
Confundir linhas e colunas	não conferir formatos	print(.shape): X (n,f), D (n,K)
Média na dimensão errada	axis=1 dá a média de cada ponto	pontos.mean(axis=0)
Não tratar cluster vazio	média de array vazio = NaN	reposicionar na amostra mais distante
Não implementar convergência	laço com número fixo de voltas	desloc < = tol → break

BLOCO 25 · ERROS MAIS COMUNS · 2/2

Causa e correção

erro	causa	correção
Esquecer `max_iter`	laço pode não terminar	for it in range(1, max_iter + 1)
Labels incompatíveis com centroides	labels de antes da última atualização	recalcular labels_ e inercia_ após o laço
Inércia como acurácia	não há resposta certa; cai sempre com K	usar só para comparar K e sementes
Índices como classes reais	«cluster 0 = baixo gasto»	interpretar pelas médias originais
Não controlar a semente	resultado muda a cada execução	random_state fixo
Escalas incompatíveis	o gasto domina a distância	(X − média) / desvio
Comparar sem controlar parâmetros	K, escala, n_init ou semente diferentes	fixar tudo, mudar uma coisa por vez

BLOCO 26 · DESAFIO FINAL

Implemente K-Means do zero e segmente clientes

01 Criar um dataset sintético
02 Implementar a inicialização
03 Calcular distâncias
04 Atribuir clusters
05 Atualizar centroides
06 Implementar convergência
07 Tratar clusters vazios
08 Calcular a inércia
09 Visualizar os grupos
10 Aplicar o método do cotovelo
11 Comparar sementes
12 Comparar com Scikit-learn
13 Interpretar os resultados

Critério de aceite: Restart Kernel → Run All sem erros, sem NaN, com labels_ e inercia_ consistentes com centroides_.

BLOCO 26 · ENTREGAS

O que entregar

- Código completo
- Classe `KMeansDoZero`
- Centroides finais
- Método do cotovelo
- Breve análise dos grupos
- Dataset utilizado
- Gráfico dos clusters
- Gráfico da evolução da inércia
- Comparação com Scikit-learn

`aula_6_10_alunos_TODO.ipynb`
estrutura e TODOs para o desafio

`aula_6_10_exercicio_guiado.ipynb`
as 18 etapas desta aula

`aula_6_10_gabarito_professor.ipynb`
solução completa e testes, liberada após o prazo

BLOCO 27 · LEITURA COMPLEMENTAR

Documentação oficial

NumPy · guia básico	numpy.org/doc/stable/user/absolute_beginners.html
NumPy · broadcasting	numpy.org/doc/stable/user/basics.broadcasting.html
Scikit-learn · KMeans	scikit-learn.org/stable/modules/generated/sklearn.cluster.KMeans.html
Scikit-learn · clusterização	scikit-learn.org/stable/modules/clustering.html
Escalonamento	scikit-learn.org/stable/modules/preprocessing.html
Matplotlib · scatter	matplotlib.org/stable/api/_as_gen/matplotlib.pyplot.scatter.html
Aprendizado não supervisionado	scikit-learn.org/stable/unsupervised_learning.html

CHECKLIST DE APRENDIZAGEM

Eu consigo...

- [ ] explicar a diferença entre K-Means e KNN
- [ ] calcular um centroide à mão
- [ ] resolver o exemplo de 4 pontos no caderno
- [ ] calcular a inércia e explicar cada símbolo
- [ ] explicar axis=0, argmin e shape
- [ ] implementar KMeansDoZero com NumPy
- [ ] vetorizar distâncias com broadcasting
- [ ] tratar clusters vazios sem NaN
- [ ] garantir labels_ consistentes com centroides_
- [ ] ler um gráfico de cotovelo com cautela
- [ ] justificar o escalonamento
- [ ] interpretar grupos pelas médias originais